

查看官方手册并分析现象

在研究问题之前先查看 `system()` 的官方手册，使用 `man` 命令来查看系统手册。值得一提的是，虽然函数名为 `system()` 但是函数本身并不是系统函数（ `system calls`）而是库函数（`library calls`）。使用 `man 3 system` 即可查看所需信息。

```
SYSTEM(3)                                Linux Programmer's Manual                                SYSTEM(3)

NAME
    system - execute a shell command

SYNOPSIS
    #include <stdlib.h>

    int system(const char *command);

DESCRIPTION
    The system() library function uses fork(2) to create a child process that executes the shell command specified in command using execl(3) as follows:

        execl("/bin/sh", "sh", "-c", command, (char *) 0);

    system() returns after the command has been completed.

    During execution of the command, SIGCHLD will be blocked, and SIGINT and SIGQUIT will be ignored, in the process that calls system() (these signals will be handled according to their defaults inside the child process that executes command).

    If command is NULL, then system() returns a status indicating whether a shell is available on the system.
```

得到了以上的信息，简单来说就是 `system()` 函数会调用 `fork()` 函数来创建子进程，在子进程中调用 `execl()` 函数来运行 `Shell` 命令。在父进程中，`SIGCHLD` 信号会被阻塞，`SIGINT` 和 `SIGQUIT` 信号会被忽视。

有了这些信息便可以解释 `test.cpp` 出现的使用 `CTRL + C` 却没有反应的事情，`CTRL + C` 对所有前台程序发送 `SIGINT` 信号，此时主进程将信号屏蔽掉，而子进程会收到信号并退出。可以做实验来验证这一点，在 `test` 程序运行时使用 `CTRL + C`，会立刻出现 `Hello World!`，这是因为子进程被终止，父进程回收子进程并结束，`system()` 函数返回，进入新的循环并打印。

实验和推测该函数的实现

首先可以验证 `system()` 函数是否通过 `fork()` 执行。这个验证比较简单，可以运行 `test` 程序，并在另一终端上查看 `pid`，找该进程是否有子进程。

```
[root@Host-1953697 ~]# ps -ef | grep test
root      2790      2047  0 00:26 pts/0    00:00:00 ./test
root      2800      2674  0 00:27 pts/1    00:00:00 grep --color=auto test
[root@Host-1953697 ~]# ps -ef | grep 2790
root      2790      2047  0 00:26 pts/0    00:00:00 ./test
root      2804      2790  0 00:27 pts/0    00:00:00 sleep 2
root      2806      2674  0 00:27 pts/1    00:00:00 grep --color=auto 2790
[root@Host-1953697 ~]# kill -9 2790
[root@Host-1953697 ~]#
```

首先，我通过 `ps` 命令查找到 `test` 的主进程 `pid` 为 2790，之后我查找到 `pid` 为 2804 的子进程，证实了这一观点，并且可以看到子进程的进程命令是 “`sleep 2`”，这正是被 `execl()` 命令更改的。

之后可以验证是否 `SIGCHLD` 信号被阻塞，这部分比较好验证，只要调用 `signal()` 函数对 `SIGCHLD` 捕获即可。测试程序如下，如果 `SIGCHLD` 真的被捕获，那在子进程执行 `sleep()` 时应该不会打印 `recv sigchld` 信息，会在执行结束后才打印。测试程序如下：

```

#include <iostream>
#include <unistd.h>
#include <stdlib.h>
#include <signal.h>
using namespace std;

void signalhandler(int sig)
{
    switch (sig)
    {
        case SIGCHLD:
            write(1, "recv sigchld\n", sizeof("recv sigchld\n")); //原子操作
            break;
    }
}

int main(int argc, char *argv[])
{
    signal(SIGCHLD, signalhandler);
    while (1)
    {
        cout << "Hello World!" << endl;
        system("sleep 2"); //主进程屏蔽SIGCHLD
    }
    return 0;
}

```

但是测试结果有些奇怪，我在 *test* 程序执行时发送了多个 *SIGCHLD* 信号，但是每次主进程只会打印一次“recv sigchld”，并且在正常运行情况下，主进程也会在子进程结束时收到 *SIGCHLD* 信号。对于这两个问题我的推测是多个 *SIGCHLD* 信号被阻塞到，解除阻塞后所有信号会一次性传给主进程，但是只会触发一次指定的信号处理程序，并且虽然 *system()* 函数应该回收了子进程的程序碎片，但是子程序退出时的信号没有被拦截而是阻塞，这便导致总会有 *SIGCHLD* 信号传入。以后使用 *system()* 函数时要注意这一点。

寻找答案和验证

有关 *system()* 函数的信息可以从《UNIX环境高级编程》这本书中得到解答，可以查看 8.13节和10.18节。其中包括了 *system* 函数的 POSIX.1 正确实现

```

#include <sys/wait.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>

int
system(const char *cmdstring)      /* with appropriate signal handling */
{
    pid_t      pid;
    int        status;
    struct sigaction ignore, saveintr, savequit;
    sigset_t    chldmask, savemask;

    if (cmdstring == NULL)
        return(1);                /* always a command processor with UNIX */

    ignore.sa_handler = SIG_IGN;   /* ignore SIGINT and SIGQUIT */
    sigemptyset(&ignore.sa_mask);
    ignore.sa_flags = 0;
    if (sigaction(SIGINT, &ignore, &saveintr) < 0)
        return(-1);
    if (sigaction(SIGQUIT, &ignore, &savequit) < 0)
        return(-1);
    sigemptyset(&chldmask);        /* now block SIGCHLD */
    sigaddset(&chldmask, SIGCHLD);
    if (sigprocmask(SIG_BLOCK, &chldmask, &savemask) < 0)
        return(-1);

    if ((pid = fork()) < 0) {
        status = -1; /* probably out of processes */
    } else if (pid == 0) {         /* child */
        /* restore previous signal actions & reset signal mask */
        sigaction(SIGINT, &saveintr, NULL);

        sigaction(SIGQUIT, &savequit, NULL);
        sigprocmask(SIG_SETMASK, &savemask, NULL);

        execl("/bin/sh", "sh", "-c", cmdstring, (char *)0);
        _exit(127);                /* exec error */
    } else {                       /* parent */
        while (waitpid(pid, &status, 0) < 0)
            if (errno != EINTR) {
                status = -1; /* error other than EINTR from waitpid() */
                break;
            }
    }

    /* restore previous signal actions & reset signal mask */
    if (sigaction(SIGINT, &saveintr, NULL) < 0)
        return(-1);
    if (sigaction(SIGQUIT, &savequit, NULL) < 0)
        return(-1);
    if (sigprocmask(SIG_SETMASK, &savemask, NULL) < 0)
        return(-1);

    return(status);
}

```

图 10-28 system 函数的 POSIX.1 正确实现

